

Вот подробный конспект семинара по Модулям 9-10. Структура адаптирована под конвертацию в `.ipynb`: теория чередуется с кодовыми ячейками, каждое задание сопровождается примером решения.

Семинар: Модули 9-10. Интеграция ML в продакшн

Часть 1. Интеграция ML в асинхронный бэкенд

1.1. Архитектурные паттерны ML-инференса

Паттерн 1: Синхронный inference в endpoint

Когда модель легкая (< 100 мс, CPU), запрос обрабатывается в рамках одного HTTP-запроса. Но `model.predict()` блокирует event loop.

```
In [1]: from fastapi import FastAPI
        from pydantic import BaseModel
        import asyncio
        from concurrent.futures import ProcessPoolExecutor

        app = FastAPI()

        # Имитация ML-модели
        class FakeModel:
            def predict(self, features: list[float]) -> list[float]:
                # Тяжелое вычисление (блокирует поток)
                import time
                time.sleep(0.1)
                return [sum(features) / len(features)]

        model = FakeModel()
        executor = ProcessPoolExecutor(max_workers=2)

        class InputModel(BaseModel):
            features: list[float]

        # ПЛОХО: блокирует event loop
        @app.post("/predict-sync/")
        async def predict_sync(input_data: InputModel):
            result = model.predict(input_data.features) # БЛОКИРУЕТ!
```

```

    return {"prediction": result}

# ХОРОШО: в ProcessPoolExecutor
@app.post("/predict-async/")
async def predict_async(input_data: InputModel):
    loop = asyncio.get_running_loop()
    result = await loop.run_in_executor(executor, model.predict, input_data.features)
    return {"prediction": result}

# uvicorn seminar_910:app --port 8000
# curl -X POST http://localhost:8000/predict-async/ -H "Content-Type: application/json" -d '{"features":[1.0,2.0,3.0]}'

```

Паттерн 2: Асинхронный inference через очередь

Для тяжелых моделей (> 500 мс): клиент получает job ID и опрашивает статус.

In [2]:

Клиент	FastAPI	Очередь	Worker
--- POST /predict ----->			
{features}			
	--- enqueue(job_id) ---->		
<-- 202 Accepted -----			
{job_id: "abc123"}			
--- GET /status/abc123 ->			
<-- 200 OK -----			
{status: "pending"}			
		--- dispatch ----->	i
nference		<-- result -----	
	<-- store result -----		
--- GET /status/abc123 ->			
<-- 200 OK -----			
{status: "done",			
result: {...}}			

Паттерн 3: Batch inference

GPU эффективнее с батчами. Динамический батчинг накапливает запросы на короткое время (10 мс), затем делает один forward pass.

```
In [3]:  Время:  0ms    5ms    10ms
        Запросы: R1 ->  R2 ->  R3 ->  [BATCH: R1+R2+R3] -> inference -> ответы всем трем
```

1.2. Теория очередей для ML

M/M/1 модель (один worker):

```
In [4]:   $\rho = \lambda / \mu = \lambda * S$  (коэффициент загрузки)
         $Wq = \rho / (\mu(1-\rho)) = \lambda * S^2 / (1-\lambda * S)$ 
```

При inference = 1 с и $\lambda = 0.99$ RPS:

```
In [5]:   $Wq = 0.99 * 1 / (1 - 0.99) = 99$  секунд!
```

M/M/c модель (с worker'ов):

```
In [6]:   $\rho = \lambda / (c * \mu)$ 
```

При $\rho = 0.5$, $c = 4$: $P_{wait} \approx 8\%$. При $\rho = 0.8$: $P_{wait} \approx 42\%$.

Практический вывод: держите $\rho < 0.7$.

Оптимальный размер батча:

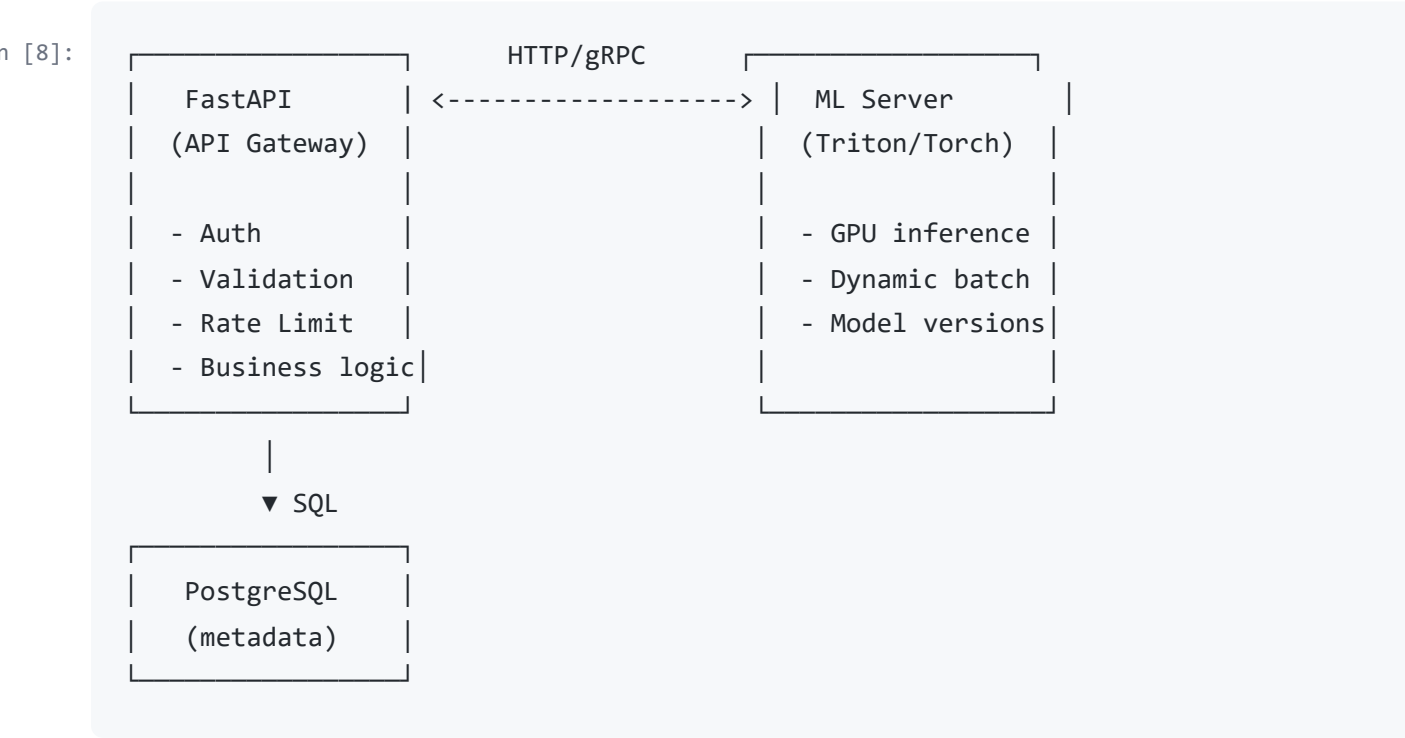
```
In [7]:  T(n) = T_fixed + n * T_per_item      (время inference для батча n)
        Latency per item = T(n) / n          (среднее время на один пример)
        W_batch = (n-1) / (2*\lambda)        (время ожидания формирования батча)
        Total(n) = W_batch + T(n)            (суммарная latency)
```

Если T_{fixed} велико и λ высока — большой батч выгоден. Если λ низкая — батчинг убивает latency.

1.3. ML-серверы и форматы моделей

Формат/Сервер	Описание	Когда использовать
ONNX Runtime	Универсальный оптимизированный движок. Удаляет мертвый код, сливает операции, использует AVX512/CUDA	Ускорение 5-20x vs чистый PyTorch
TensorRT	NVIDIA-специфичный. Mixed precision FP16/INT8, Tensor Cores	Только NVIDIA GPU, максимальная скорость
OpenVINO	Intel-специфичный. VNNI для INT8	Intel CPU/GPU, edge-устройства
TorchServe	Официальный сервер PyTorch. MAR-архивы, REST/gRPC, batching, A/B testing	Экосистема PyTorch
Triton Inference Server	Промышленный стандарт NVIDIA. Dynamic batching, ensemble, GPU sharing	Production GPU-инференс
BentoML	Python-native. Генерирует Docker, adaptive traffic control	Быстрый старт, Python-first

Рекомендуемая архитектура:



FastAPI не делает inference. Он валидирует, аутентифицирует, преобразует формат, отправляет в ML-сервер, обогащает результат, записывает в БД.

1.4. Фоновые задачи в FastAPI

BackgroundTasks: простые операции в том же процессе.

```
In [9]: from fastapi import FastAPI, BackgroundTasks
import asyncio

app = FastAPI()

async def send_notification(email: str, message: str):
    await asyncio.sleep(1) # имитация отправки email
    print(f"Notification sent to {email}: {message}")

@app.post("/predict/")
async def predict(
    input_data: dict,
    background_tasks: BackgroundTasks
):
    # Основной ответ отправляется сразу
    result = {"prediction": 0.95}

    # Уведомление отправляется в фоне
    background_tasks.add_task(send_notification, "user@example.com", "Done")

    return result

# Ограничения:
# - Тот же процесс
# - При рестарте – задачи теряются
# - Нет retry, нет persistence
# - Не для долгих задач (> нескольких секунд)
```

Celery: классика распределенных задач.

```
In [10]: # Имитация Celery для семинара (без реального брокера)
class FakeCeleryTask:
    def __init__(self, func):
        self.func = func
        self.id = None
```

```

def delay(self, *args, **kwargs):
    import uuid
    self.id = str(uuid.uuid4())
    print(f"[Celery] Task {self.id} queued: {self.func.__name__}({args}, {kwargs})")
    return self

class FakeCelery:
    def task(self, bind=False, max_retries=3):
        def decorator(func):
            return FakeCeleryTask(func)
        return decorator

    class AsyncResult:
        def __init__(self, task_id):
            self.id = task_id
            self.status = "PENDING"

celery_app = FakeCelery()

@celery_app.task(bind=True, max_retries=3)
def heavy_inference(self, model_id: str, features: list):
    print(f"Running inference with {model_id}")
    return {"status": "success", "result": [0.1, 0.9]}

# Запуск из FastAPI
def predict_async(input_data: dict):
    task = heavy_inference.delay("model_v1", input_data.get("features", []))
    return {"task_id": task.id, "status": "queued"}

# Демонстрация
result = predict_async({"features": [1.0, 2.0]})
print(result)

```

Современные альтернативы:

- **arq**: async-native, на Redis Streams. Worker'ы — async.
- **dramatiq**: простой, reliable, middleware (rate limiting, retries).
- **saq**: минималистичный async worker с cron.

1.5. Поточковая обработка

StreamingResponse для генеративных моделей:

```
In [11]: from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import asyncio

app = FastAPI()

async def token_streamer(prompt: str):
    tokens = ["The", " future", " of", " AI", " is", " bright", "."]
    for token in tokens:
        await asyncio.sleep(0.3) # имитация inference
        yield token

@app.post("/generate/")
async def generate(prompt: str):
    return StreamingResponse(
        token_streamer(prompt),
        media_type="text/plain",
    )

# curl -X POST "http://localhost:8000/generate/?prompt=hello"
# Токены приходят по мере генерации
```

Server-Sent Events (SSE):

```
In [12]: import json
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

async def progress_streamer(task_id: str):
    for progress in range(0, 101, 20):
        await asyncio.sleep(0.5)
        yield f"data: {json.dumps({'task_id': task_id, 'progress': progress})}\n\n"
    yield f"data: {json.dumps({'task_id': task_id, 'status': 'done'})}\n\n"

@app.get("/progress/{task_id}")
async def progress(task_id: str):
    return StreamingResponse(
        progress_streamer(task_id),
        media_type="text/event-stream",
    )
```

```
# curl http://localhost:8000/progress/task_123
# События приходят автоматически
```

WebSocket для real-time:

```
In [13]: from fastapi import FastAPI, WebSocket

app = FastAPI()

@app.websocket("/ws/transcribe")
async def transcribe(websocket: WebSocket):
    await websocket.accept()
    try:
        while True:
            audio_chunk = await websocket.receive_text() # имитация
            # text = await speech_model.transcribe_chunk(audio_chunk)
            await asyncio.sleep(0.1)
            await websocket.send_text(f"Transcribed: {audio_chunk}")
    except Exception:
        await websocket.close()

# Latency budget для real-time:
# Сеть: 20-50 мс | Десериализация: 1-5 мс | Предобработка: 5-20 мс
# Inference: 30-100 мс | Постобработка: 1-5 мс | Сеть обратно: 10-30 мс
# Итого: 70-210 мс (для интерактивного опыта < 300 мс)
```

1.6. Feature Store и кэширование фичей

Тип фичей	Примеры	Источник	Latency
Online	Геолокация, время суток, последний клик	Запрос, Kafka, Redis	< 10 мс
Offline	История покупок за год, агрегаты	Data Warehouse, S3	Предвычислены
Near-real-time	Счетчик просмотров за час	Flink, Kafka Streams	< 100 мс

```
In [14]: # Кэширование фичей в Redis

class FakeRedis:
```



```

def __init__(self):
    self._data = {}

    async def get(self, key: str):
        return self._data.get(key)

    async def setex(self, key: str, seconds: int, value: str):
        self._data[key] = value

redis = FakeRedis()

async def get_features(user_id: str) -> dict:
    cached = await redis.get(f"features:{user_id}")
    if cached:
        return {"cached": True, "data": eval(cached)}

    # Имитация вычисления из DWH
    features = {"avg_purchase": 150.0, "category_pref": "electronics"}
    await redis.setex(f"features:{user_id}", 300, str(features))
    return {"cached": False, "data": features}

async def demo_features():
    r1 = await get_features("user_123")
    print(f"First: {r1}")
    r2 = await get_features("user_123")
    print(f"Second: {r2}")

asyncio.run(demo_features())

```

Covariate shift: модель обучена на $P_{\text{train}}(X)$, но на инференсе $P_{\text{inference}}(X)$ может отличаться. Мониторинг через Wasserstein distance.

Часть 2. Продакшн, инфраструктура и DevOps

2.1. Контейнеризация

Namespaces (изоляция): PID, NET, MNT, USER. **cgroups** (ограничение): CPU, память, I/O, pids.

```

In [15]: # ===== ЭТАП 1: Сборка =====
FROM python:3.12-slim AS builder

WORKDIR /app

```

```

RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc libpq-dev \
    && rm -rf /var/lib/apt/lists/*

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# ===== ЭТАП 2: Выполнение =====
FROM python:3.12-slim AS runtime

WORKDIR /app

COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

RUN groupadd -r appgroup && useradd -r -g appgroup appuser
USER appuser

COPY --chown=appuser:appgroup ./app ./app

HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Кэширование слоев: размещайте от редко изменяемого к часто изменяемому.

```

In [16]: # ХОРОШО: зависимости кэшируются
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY ./app ./app # только этот слой пересобирается

```

Базовые образы:

Образ	Размер	Когда
python:3.12	~900 МБ	Сборка, отладка
python:3.12-slim	~120 МБ	Стандарт production
python:3.12-alpine	~55 МБ	Минимальный, но musl ломает wheels
gcr.io/distroless/python3	~50 МБ	Максимальная безопасность, нет shell

2.2. Конфигурация

```
In [17]: from pydantic_settings import BaseSettings, SettingsConfigDict
from pydantic import PostgresDsn, RedisDsn, Field

class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        case_sensitive=False,
    )

    database_url: PostgresDsn
    redis_url: RedisDsn
    secret_key: str = Field(min_length=32)
    debug: bool = False
    log_level: str = "INFO"
    max_workers: int = Field(default=4, ge=1, le=64)

# settings = Settings() # fail-fast: не запустится без DATABASE_URL
```

The Twelve-Factor App:

- **III. Config:** в env vars, никогда в коде.
- **V. Build, release, run:** build = immutable artifact, release = artifact + config, run = только запуск.

Хранение секретов:

Уровень	Пример
1. Env vars	SECRET_KEY=abc

Уровень	Пример
2. Docker secrets	<code>/run/secrets/db_password</code>
3. Kubernetes secrets	<code>kubectl create secret</code>
4. Vault	HashiCorp Vault, AWS Secrets Manager

2.3. Observability: логи, метрики, трейсы

Структурированные логи:

```
In [18]: import json
import uuid
from contextvars import ContextVar

correlation_id: ContextVar[str] = ContextVar("correlation_id", default="unknown")

def log_event(event: str, **kwargs):
    entry = {
        "timestamp": "2024-01-15T14:32:01Z",
        "level": "info",
        "event": event,
        "request_id": correlation_id.get(),
        **kwargs
    }
    print(json.dumps(entry))

# Пример
correlation_id.set(str(uuid.uuid4()))
log_event("prediction_made", model_version="v2", latency_ms=45, user_id="123")
```

Prometheus метрики:

Тип	Смысл	Пример
Counter	Моноotonно растёт	Число предсказаний
Gauge	Текущее значение	Размер очереди
Histogram	Распределение по корзинам	Latency inference

```

In [19]: # Имитация Prometheus метрик
class Counter:
    def __init__(self, name: str, labels: list[str]):
        self.name = name
        self.labels = labels
        self._values = {}

    def labels(self, **kwargs):
        key = tuple(sorted(kwargs.items()))
        if key not in self._values:
            self._values[key] = 0
        return self

    def inc(self):
        self._values[list(self._values.keys())[-1]] += 1

class Histogram:
    def __init__(self, name: str, labels: list[str], buckets: list[float]):
        self.name = name
        self.buckets = buckets
        self._observations = []

    def labels(self, **kwargs):
        return self

    def observe(self, value: float):
        self._observations.append(value)

PREDICTION_COUNTER = Counter("ml_predictions_total", ["model_version", "status"])
PREDICTION_LATENCY = Histogram("ml_prediction_duration_seconds", ["model_version"],
                                [0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0, 10.0])

# PromQL:
# rate(ml_predictions_total[5m])                # RPS
# histogram_quantile(0.95, ...)                  # p95
# rate(errors[5m]) / rate(total[5m])             # error rate

```

Трейсы (OpenTelemetry):

```

In [20]: Trace: predict-request-123
|— Span: api-gateway (0ms - 5ms)
|— Span: auth-service (5ms - 15ms)

```

```
└─ Span: fastapi-predict (15ms - 120ms)
  └─ Span: db-query (20ms - 35ms)
    └─ Span: redis-cache (35ms - 36ms, cache miss)
      └─ Span: ml-inference (40ms - 110ms)
└─ Span: postprocess (110ms - 120ms)
```

Health checks:

Проверка	Значение	Действие при провале
Liveness	"Приложение живо"	Перезапуск контейнера
Readiness	"Готово принимать трафик"	Удаление из балансировки
Startup	"Инициализация завершена"	Ждать, не проверять liveness/readiness

2.4. Масштабирование

Gunicorn + Uvicorn:

```
In [21]: gunicorn app.main:app \
         -w 4 \
         -k uvicorn.workers.UvicornWorker \
         --bind 0.0.0.0:8000 \
         --timeout 120
```

Сколько worker'ов?

- Синхронные: $N = 2 * C + 1$
- Асинхронные (Uvicorn): $N = C$ (иногда $C + 1$)

WebSocket масштабирование:

- **Sticky Sessions:** балансировщик "приклеивает" клиента к серверу по IP.
- **Redis Pub/Sub:** broadcast между серверами.

2.5. CI/CD

Rolling Deployment: постепенная замена. Просто, но дефектная версия затрагивает часть пользователей.

Blue-Green: два окружения, мгновенное переключение. Вдвое больше ресурсов, но мгновенный откат.

Canary: 5% -> 50% -> 100%. Автоматический откат при деградации метрик.

```
In [22]: Z = (p1 - p2) / sqrt(p*(1-p)*(1/n1 + 1/n2))
|Z| > 1.96 => статистически значимая разница => откат
```

Миграции в CI/CD:

- Применять **до** деплоя.
- Использовать `CREATE INDEX CONCURRENTLY`.
- Expand/Contract для zero-downtime.

Задания для самостоятельной работы

Задание 1. Синхронный vs асинхронный inference

Создайте endpoint, который "предсказывает" результат через `asyncio.sleep(0.5)`. Сделайте две версии: одну, которая блокирует event loop (плохо), и одну через `run_in_executor` (хорошо). Замерьте время обработки 3 конкурентных запросов.

```
In [23]: import asyncio
from concurrent.futures import ProcessPoolExecutor
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class InputModel(BaseModel):
    features: list[float]

executor = ProcessPoolExecutor(max_workers=2)

def heavy_predict(features: list[float]) -> float:
    import time
    time.sleep(0.5) # имитация блокирующего inference
    return sum(features) / len(features)

# ПЛОХО: блокирует event loop
@app.post("/predict-bad/")
async def predict_bad(input_data: InputModel):
```

```

    result = heavy_predict(input_data.features) # БЛОКИРУЕТ!
    return {"prediction": result}

# ХОРОШО: в ProcessPool
@app.post("/predict-good/")
async def predict_good(input_data: InputModel):
    loop = asyncio.get_running_loop()
    result = await loop.run_in_executor(executor, heavy_predict, input_data.features)
    return {"prediction": result}

# Тест конкурентности
async def test_concurrent():
    import time
    import httpx

    async with httpx.AsyncClient(app=app, base_url="http://test") as client:
        # Тест плохой версии
        start = time.perf_counter()
        responses = await asyncio.gather(*[
            client.post("/predict-bad/", json={"features": [1.0, 2.0]})
            for _ in range(3)
        ])
        bad_time = time.perf_counter() - start
        print(f"Bad version: {bad_time:.2f}s (ожидалось ~1.5s, т.к. последовательнo)")

        # Тест хорошей версии
        start = time.perf_counter()
        responses = await asyncio.gather(*[
            client.post("/predict-good/", json={"features": [1.0, 2.0]})
            for _ in range(3)
        ])
        good_time = time.perf_counter() - start
        print(f"Good version: {good_time:.2f}s (ожидалось ~0.5s, т.к. параллельно)")

# asyncio.run(test_concurrent())

```

Задание 2. Имитация очереди задач

Напишите in-memory "очередь", где endpoint `POST /predict/async` ставит задачу в очередь и возвращает `job_id`. Endpoint `GET /status/{job_id}` возвращает статус. Worker обрабатывает задачи в фоне.


```
In [24]: import asyncio
import uuid
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class InputModel(BaseModel):
    features: list[float]

# In-memory очередь и результаты
_job_queue = asyncio.Queue()
_job_results = {}

async def worker():
    while True:
        job_id, features = await _job_queue.get()
        # Имитация inference
        await asyncio.sleep(2)
        result = sum(features) / len(features)
        _job_results[job_id] = {"status": "done", "result": result}
        _job_queue.task_done()

# Запускаем worker при старте
@app.on_event("startup")
async def startup():
    asyncio.create_task(worker())

@app.post("/predict/async/")
async def predict_async(input_data: InputModel):
    job_id = str(uuid.uuid4())
    _job_results[job_id] = {"status": "pending"}
    await _job_queue.put((job_id, input_data.features))
    return {"job_id": job_id, "status": "queued"}

@app.get("/status/{job_id}")
async def get_status(job_id: str):
    result = _job_results.get(job_id, {"status": "not_found"})
    return {"job_id": job_id, **result}

# uvicorn seminar_910:app --port 8000
# curl -X POST http://localhost:8000/predict/async/ -H "Content-Type: application/json" -d '{"features":[1.0,2.0,3.0]}'
# curl http://localhost:8000/status/<job_id>
```

Задание 3. StreamingResponse для генерации токенов

Напишите endpoint `/generate/`, который возвращает `StreamingResponse`. Генератор выдает "токены" с задержкой 0.3 секунды между ними.

```
In [25]: from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import asyncio

app = FastAPI()

async def token_streamer(prompt: str):
    tokens = ["The", " quick", " brown", " fox", " jumps", " over", " the", " laz",
y", " dog", "."]
    for token in tokens:
        await asyncio.sleep(0.3)
        yield token

@app.post("/generate/")
async def generate(prompt: str = "hello"):
    return StreamingResponse(
        token_streamer(prompt),
        media_type="text/plain",
    )

# uvicorn seminar_910:app --port 8000
# curl -X POST "http://localhost:8000/generate/?prompt=hello" --no-buffer
# Токены приходят по одному
```

Задание 4. SSE для прогресса задачи

Напишите endpoint `/progress/{task_id}`, который возвращает SSE с прогрессом выполнения задачи (0%, 20%, ..., 100%).

```
In [26]: import json
import asyncio
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

async def progress_streamer(task_id: str):
    for progress in range(0, 101, 20):
```

```

        await asyncio.sleep(0.5)
        yield f"data: {json.dumps({'task_id': task_id, 'progress': progress})}\n"
    \n"
    yield f"data: {json.dumps({'task_id': task_id, 'status': 'done'})}\n\n"

@app.get("/progress/{task_id}")
async def progress(task_id: str):
    return StreamingResponse(
        progress_streamer(task_id),
        media_type="text/event-stream",
    )

# uvicorn seminar_910:app --port 8000
# curl http://localhost:8000/progress/task_123
# События приходят автоматически каждые 0.5 сек

```

Задание 5. Dockerfile с многоэтапной сборкой

Напишите Dockerfile для FastAPI-приложения с многоэтапной сборкой. Этап builder устанавливает зависимости, этап runtime — только копирует venv и код. Добавьте непривилегированного пользователя и HEALTHCHECK.

```

In [27]: # ===== ЭТАП 1: Сборка =====
FROM python:3.12-slim AS builder

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
    gcc \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# ===== ЭТАП 2: Выполнение =====
FROM python:3.12-slim AS runtime

```

```

WORKDIR /app

COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

RUN groupadd -r appgroup && useradd -r -g appgroup appuser
USER appuser

COPY --chown=appuser:appgroup ./app ./app

HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Задание 6. Pydantic Settings с валидацией

Напишите класс `Settings` через `pydantic-settings`, который валидирует `DATABASE_URL` (должен быть PostgreSQL URL), `SECRET_KEY` (минимум 32 символа) и `MAX_WORKERS` (от 1 до 64). Продемонстрируйте fail-fast при отсутствии обязательных переменных.

```

In [28]: from pydantic_settings import BaseSettings, SettingsConfigDict
        from pydantic import PostgresDsn, Field, ValidationError

        class Settings(BaseSettings):
            model_config = SettingsConfigDict(
                env_file=".env",
                env_file_encoding="utf-8",
                case_sensitive=False,
            )

            database_url: PostgresDsn
            secret_key: str = Field(min_length=32)
            debug: bool = False
            max_workers: int = Field(default=4, ge=1, le=64)

        # Тест с корректными данными
        try:
            settings = Settings(
                DATABASE_URL="postgresql+asyncpg://user:pass@localhost/db",
                SECRET_KEY="a" * 32,
            )

```

```

        MAX_WORKERS=8,
    )
    print(f"Settings loaded: db={settings.database_url}, workers={settings.max_workers}")
except ValidationError as e:
    print(e)

# Тест с ошибками
try:
    bad_settings = Settings(
        DATABASE_URL="invalid-url",
        SECRET_KEY="short",
        MAX_WORKERS=100,
    )
except ValidationError as e:
    print(f"Validation failed (expected):")
    for err in e.errors():
        print(f"  {err['loc']}: {err['msg']}")

```

Задание 7. Структурированные логи с correlation ID

Напишите middleware, которая генерирует `X-Request-ID` для каждого запроса и включает его в логи. Используйте `ContextVar` для передачи ID через корутины.

```

In [29]: import json
import uuid
from contextvars import ContextVar
from fastapi import FastAPI, Request

app = FastAPI()

request_id_var: ContextVar[str] = ContextVar("request_id", default="unknown")

def log_event(event: str, **kwargs):
    entry = {
        "timestamp": "2024-01-15T14:32:01Z",
        "level": "info",
        "event": event,
        "request_id": request_id_var.get(),
        **kwargs
    }
    print(json.dumps(entry))

@app.middleware("http")

```

```

async def add_correlation_id(request: Request, call_next):
    req_id = request.headers.get("x-request-id", str(uuid.uuid4()))
    request_id_var.set(req_id)

    response = await call_next(request)
    response.headers["x-request-id"] = req_id
    return response

@app.get("/api/")
async def api_endpoint():
    log_event("request_received", endpoint="/api/")
    return {"status": "ok"}

# uvicorn seminar_910:app --port 8000
# curl -H "X-Request-ID: my-custom-id" http://localhost:8000/api/
# Логи содержат request_id

```

Задание 8. Имитация Prometheus метрик

Напишите классы `Counter` и `Histogram`, которые имитируют Prometheus. Endpoint `/predict/` должен инкрементировать счетчик и записывать latency. Endpoint `/metrics/` возвращает текущие значения.

```

In [30]: import time
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Counter:
    def __init__(self, name: str, labels: list[str]):
        self.name = name
        self._values = {}

    def labels(self, **kwargs):
        key = tuple(sorted(kwargs.items()))
        if key not in self._values:
            self._values[key] = 0
        self._current_key = key
        return self

    def inc(self):
        self._values[self._current_key] += 1

```

```

    def to_dict(self):
        return {str(k): v for k, v in self._values.items()}

class Histogram:
    def __init__(self, name: str, labels: list[str], buckets: list[float]):
        self.name = name
        self.buckets = buckets
        self._observations = []

    def labels(self, **kwargs):
        return self

    def observe(self, value: float):
        self._observations.append(value)

    def to_dict(self):
        return {
            "count": len(self._observations),
            "sum": sum(self._observations),
            "buckets": {b: sum(1 for o in self._observations if o <= b) for b in
self.buckets}
        }

PREDICTION_COUNTER = Counter("ml_predictions_total", ["model_version", "status"])
PREDICTION_LATENCY = Histogram("ml_prediction_duration_seconds", ["model_versio
n"],
                                [0.01, 0.05, 0.1, 0.5, 1.0, 5.0])

class InputModel(BaseModel):
    features: list[float]

@app.post("/predict/")
async def predict(input_data: InputModel):
    start = time.perf_counter()
    try:
        # Имитация inference
        await asyncio.sleep(0.1)
        result = {"prediction": sum(input_data.features) / len(input_data.feature
s)}

        PREDICTION_COUNTER.labels(model_version="v2", status="success").inc()
        return result
    except Exception:
        PREDICTION_COUNTER.labels(model_version="v2", status="error").inc()
        raise
    finally:
        PREDICTION_LATENCY.labels(model_version="v2").observe(time.perf_counter())

```

```
- start)

@app.get("/metrics/")
async def metrics():
    return {
        "predictions": PREDICTION_COUNTER.to_dict(),
        "latency": PREDICTION_LATENCY.to_dict(),
    }

# uvicorn seminar_910:app --port 8000
# curl -X POST http://localhost:8000/predict/ -H "Content-Type: application/json"
# -d '{"features":[1.0,2.0,3.0]}'
# curl http://localhost:8000/metrics/
```

Задание 9. Health checks (liveness + readiness)

Напишите endpoint'ы `/health/live` и `/health/ready`. Readiness проверяет "подключение" к БД (имитация: 90% успех). Liveness всегда возвращает OK.

```
In [31]: import random
from fastapi import FastAPI, HTTPException

app = FastAPI()

# Имитация состояния подключений
_db_connected = True
_redis_connected = True

@app.get("/health/live")
async def liveness():
    return {"status": "alive"}

@app.get("/health/ready")
async def readiness():
    # Имитация: иногда БД недоступна
    if random.random() < 0.1: # 10% шанс "проблемы"
        raise HTTPException(status_code=503, detail="Database temporarily unavailable")

    if not _db_connected:
        raise HTTPException(status_code=503, detail="Database unavailable")
    if not _redis_connected:
        raise HTTPException(status_code=503, detail="Redis unavailable")
```



```

    return {"status": "ready"}

# uvicorn seminar_910:app --port 8000
# curl http://localhost:8000/health/live # всегда 200
# curl http://localhost:8000/health/ready # обычно 200, иногда 503

```

Задание 10. Token Bucket Rate Limiter

Реализуйте `TokenBucket` и интегрируйте в FastAPI. Endpoint `/api/` принимает запросы только при наличии токенов.

```

In [32]: import time
        from fastapi import FastAPI, HTTPException

        app = FastAPI()

        class TokenBucket:
            def __init__(self, capacity: int, rate: float):
                self.capacity = capacity
                self.rate = rate
                self.tokens = float(capacity)
                self.last_update = time.monotonic()

            def consume(self, tokens: int = 1) -> bool:
                now = time.monotonic()
                elapsed = now - self.last_update
                self.tokens = min(self.capacity, self.tokens + elapsed * self.rate)
                self.last_update = now

                if self.tokens >= tokens:
                    self.tokens -= tokens
                    return True
                return False

        bucket = TokenBucket(capacity=5, rate=2) # 5 burst, 2/сек

        @app.get("/api/")
        async def api_endpoint():
            if not bucket.consume():
                raise HTTPException(status_code=429, detail="Too many requests")
            return {"message": "Success", "remaining_tokens": round(bucket.tokens, 2)}

# uvicorn seminar_910:app --port 8000
# for i in {1..10}; do curl http://localhost:8000/api/; echo; done

```

Задание 11. Docker Compose для локальной разработки

Напишите `docker-compose.yml` с сервисами: `api` (FastAPI), `db` (PostgreSQL), `redis` (Redis).
API зависит от db и redis, использует volumes для hot-reload.

```
In [33]: # docker-compose.yml
version: "3.9"

services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql+asyncpg://user:pass@db:5432/ml_db
      - REDIS_URL=redis://redis:6379
    depends_on:
      - db
      - redis
    volumes:
      - ./app:/app/app # hot-reload для разработки

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: ml_db
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

volumes:
  postgres_data:
```

```
In [34]: # Команды:
# docker-compose up --build      # сборка и запуск
# docker-compose up -d          # в фоне
```

```
# docker-compose logs -f api      # логи сервиса api
# docker-compose down -v          # остановка + удаление volumes
```

Задание 12. Полный pipeline: очередь + SSE + метрики

Создайте endpoint `POST /analyze/`, который ставит задачу анализа в очередь. Endpoint `/progress/{job_id}` возвращает SSE с прогрессом. По завершении результат доступен через `/result/{job_id}`. Добавьте счетчик выполненных задач.

```
In [35]: import asyncio
import uuid
import json
import time
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

# Очередь и хранилище
_task_queue = asyncio.Queue()
_task_progress = {} # job_id -> {"progress": int, "status": str}
_task_results = {}
_completed_count = 0

async def analysis_worker():
    global _completed_count
    while True:
        job_id = await _task_queue.get()
        _task_progress[job_id] = {"progress": 0, "status": "running"}

        for progress in range(10, 101, 10):
            await asyncio.sleep(0.3) # имитация работы
            _task_progress[job_id] = {"progress": progress, "status": "running"}

        _task_results[job_id] = {"result": f"Analysis complete for {job_id}"}
        _task_progress[job_id] = {"progress": 100, "status": "done"}
        _completed_count += 1
        _task_queue.task_done()

@app.on_event("startup")
async def startup():
    for _ in range(2): # 2 worker'a
        asyncio.create_task(analysis_worker())
```

```

@app.post("/analyze/")
async def analyze():
    job_id = str(uuid.uuid4())
    _task_progress[job_id] = {"progress": 0, "status": "queued"}
    await _task_queue.put(job_id)
    return {"job_id": job_id, "status": "queued"}

@app.get("/progress/{job_id}")
async def progress_stream(job_id: str):
    async def event_stream():
        while True:
            progress = _task_progress.get(job_id, {"progress": 0, "status": "unknown"})
            yield f"data: {json.dumps(progress)}\n\n"
            if progress["status"] in ("done", "error"):
                break
            await asyncio.sleep(0.5)

    return StreamingResponse(event_stream(), media_type="text/event-stream")

@app.get("/result/{job_id}")
async def get_result(job_id: str):
    result = _task_results.get(job_id)
    if not result:
        return {"job_id": job_id, "status": "not_ready"}
    return {"job_id": job_id, **result}

@app.get("/metrics/")
async def metrics():
    return {"completed_tasks": _completed_count, "queue_size": _task_queue.qsize()}

# uvicorn seminar_910:app --port 8000
# curl -X POST http://localhost:8000/analyze/
# curl http://localhost:8000/progress/<job_id>
# curl http://localhost:8000/result/<job_id>
# curl http://localhost:8000/metrics/

```

Итоговая сводка

Концепция	Модуль	Ключевой API / Паттерн
Синхронный inference	9	<code>run_in_executor</code> для CPU-bound

Концепция	Модуль	Ключевой API / Паттерн
Асинхронный inference	9	Очередь + job ID + опрос статуса
Batch inference	9	Динамический батчинг, Triton
ONNX Runtime	9	Универсальный оптимизированный движок
TensorRT	9	NVIDIA-specific, mixed precision
Triton	9	Dynamic batching, ensemble, GPU sharing
BackgroundTasks	9	<code>add_task</code> , тот же процесс, нет persistence
Celery	9	Брокер + worker + backend, retry, Flower
arq/saq	9	Async-native альтернативы Celery
StreamingResponse	9	Token-by-token генерация
SSE	9	<code>text/event-stream</code> , auto-reconnect
WebSocket	9	Дуплекс real-time, sticky sessions
Feature Store	9	Online/offline фичи, Feast, Redis кэш
Covariate shift	9	Wasserstein distance, мониторинг
Namespaces/cgroups	10	PID, NET, MNT изоляция
Multi-stage build	10	builder + runtime, минимальный образ
Кэш слоев Docker	10	От редкого к частому
Distroless	10	Без shell, максимальная безопасность
Pydantic Settings	10	Валидация env vars, fail-fast
Twelve-Factor App	10	Config в env, build/release/run
Vault	10	Динамические секреты с TTL
Structlog	10	JSON-логи, correlation ID
Prometheus	10	Counter, Gauge, Histogram, Summary
histogram_quantile	10	Линейная интерполяция в buckets

Концепция	Модуль	Ключевой API / Паттерн
OpenTelemetry	10	Traces, spans, traceparent
Liveness/Readiness	10	/health/live, /health/ready, startup probe
Gunicorn + Uvicorn	10	<code>-w C</code> , не <code>2*C+1</code> для async
WebSocket scaling	10	Sticky sessions, Redis Pub/Sub
Закон Литтла	10	$L = \lambda * W$, capacity planning
Rolling/Blue-Green/Canary	10	Стратегии деплоя
Expand/Contract	10	Zero-downtime миграции
CI/CD pipeline	10	Lint -> Test -> Build -> Deploy

Чек-лист для самопроверки

- ☐ Я знаю, почему `model.predict()` в `async def` блокирует event loop.
- ☐ Я могу выбрать между синхронным, асинхронным и batch inference.
- ☐ Я понимаю модель M/M/1 и почему $\rho \rightarrow 1$ взрывает latency.
- ☐ Я знаю разницу между ONNX Runtime, TensorRT и OpenVINO.
- ☐ Я понимаю, когда использовать BackgroundTasks, а когда — Celery/arq.
- ☐ Я могу реализовать StreamingResponse и SSE.
- ☐ Я понимаю разницу между online и offline фичами.
- ☐ Я знаю, что такое namespaces и cgroups.
- ☐ Я могу написать многоэтапный Dockerfile.
- ☐ Я понимаю принцип fail-fast в Pydantic Settings.
- ☐ Я знаю, зачем нужны correlation ID в логах.
- ☐ Я понимаю разницу между Counter, Gauge и Histogram.
- ☐ Я могу объяснить, зачем разделять liveness и readiness.
- ☐ Я знаю, сколько Uvicorn worker'ов нужно для async-приложения.
- ☐ Я понимаю стратегии Rolling, Blue-Green и Canary deployment.
- ☐ Я знаю, почему миграции применяются до деплоя.